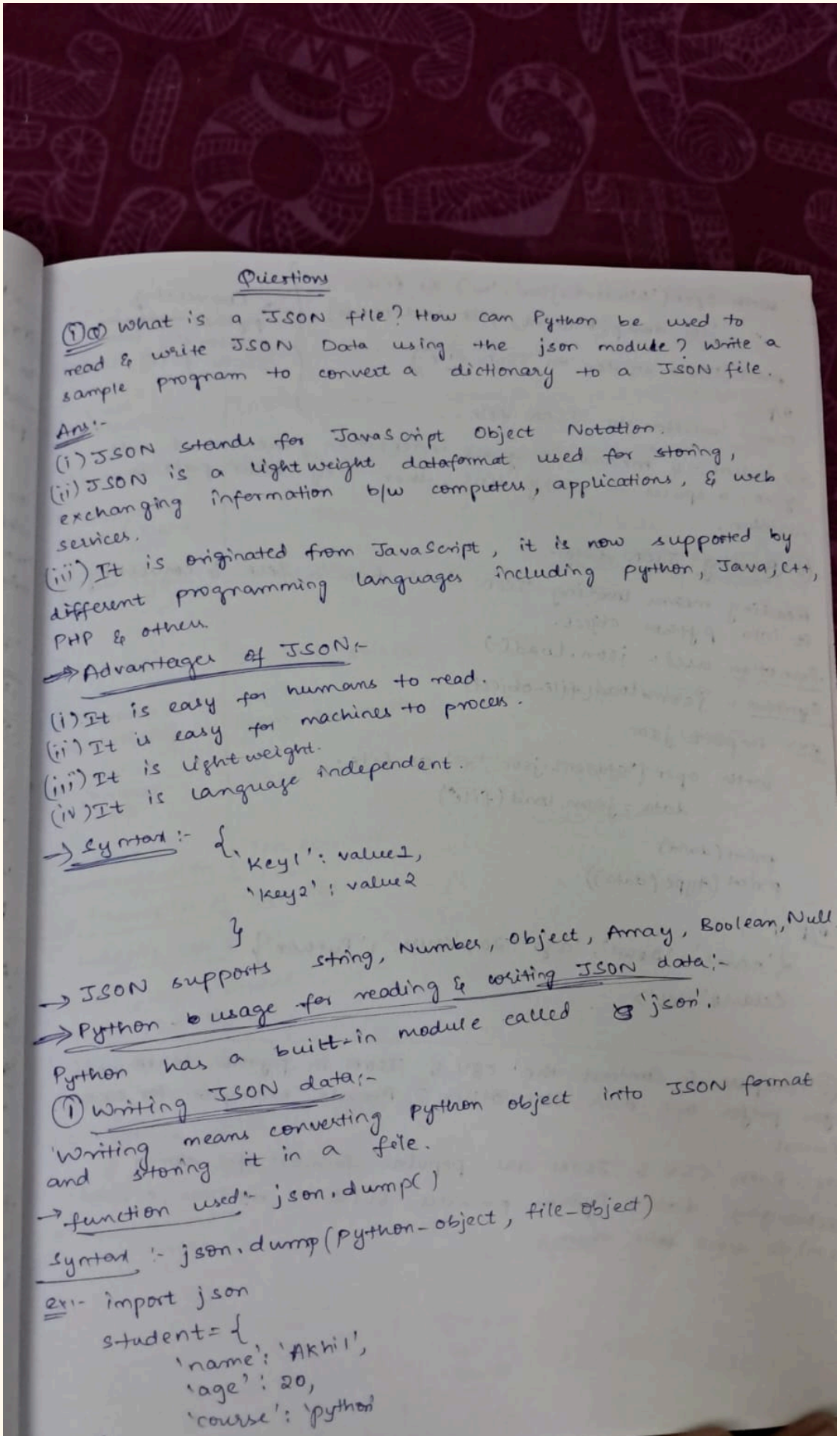


Questions

(i)(ii) What is a JSON file? How can Python be used to read & write JSON Data using the json module? Write a sample program to convert a dictionary to a JSON file.

Ans:-

(i) JSON stands for JavaScript Object Notation.

(ii) JSON is a lightweight dataformat used for storing, exchanging information b/w computers, applications, & web services.

(iii) It is originated from JavaScript, it is now supported by different programming languages including Python, Java, C++, PHP & others.

⇒ Advantages of JSON:-

(i) It is easy for humans to read.

(ii) It is easy for machines to process.

(iii) It is lightweight.

(iv) It is language independent.

→ Syntax:-

```
{
    'key1': value1,
    'key2': value2
}
```

→ JSON supports string, Number, object, Array, Boolean, Null

⇒ Python usage for reading & writing JSON data:-

Python has a built-in module called 'json'.

(i) Writing JSON data:-

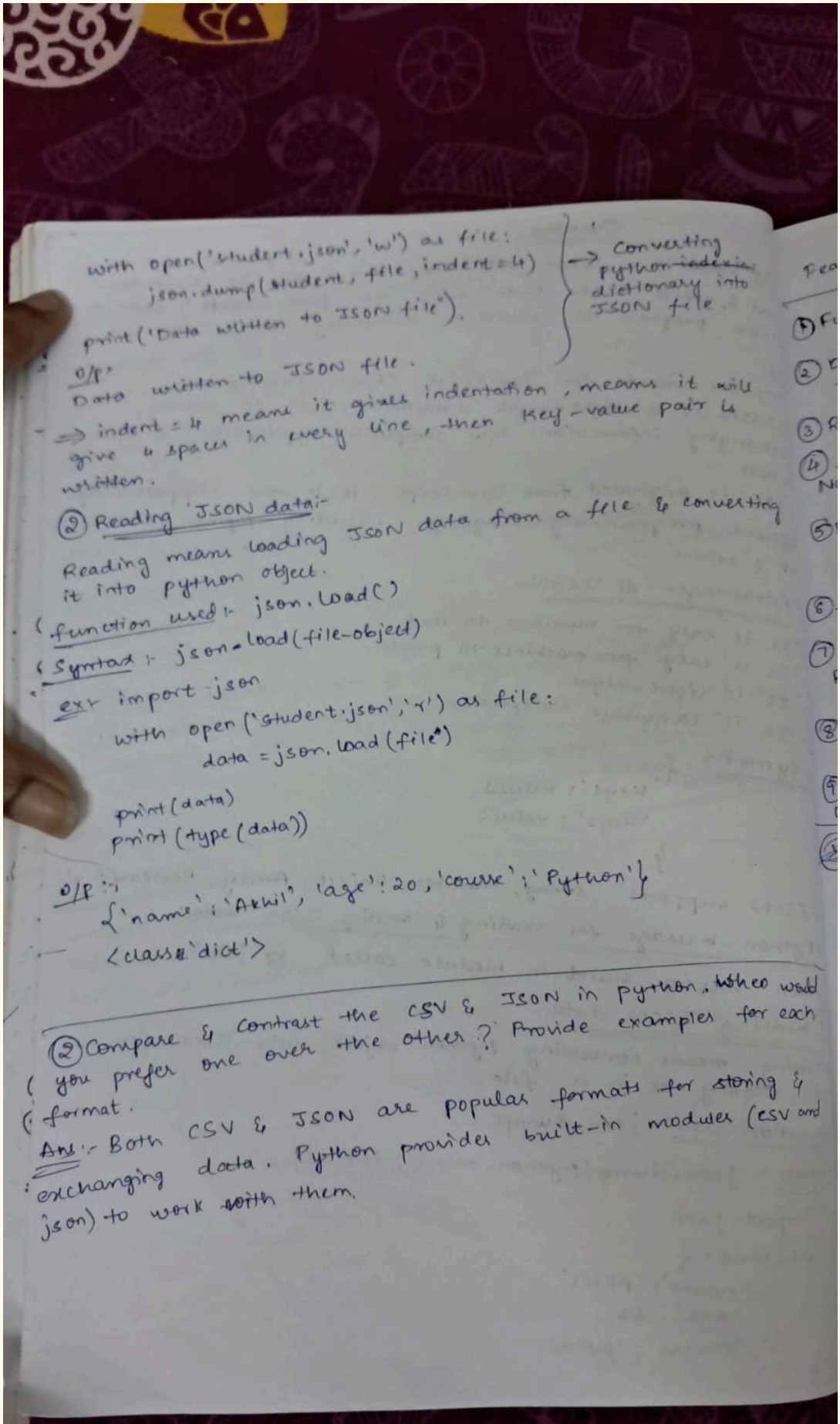
'Writing' means converting python object into JSON format and storing it in a file.

→ function used:- json.dump()

Syntax:- json.dump(python-object, file-object)

Ex:-

```
import json
student = {
    'name': 'Akhil',
    'age': 20,
    'course': 'python'
```



```
with open('student.json', 'w') as file:
    json.dump(student, file, indent=4)
print('Data written to JSON file').
```

→ Converting python dictionary into JSON file.

O/P:
Data written to JSON file.

⇒ indent = 4 means it gives indentation, means it will give 4 spaces in every line, then key-value pair is written.

② Reading JSON data:-

Reading means loading JSON data from a file & converting it into python object.

(function used :- json.load()

(Syntax :- json.load(file-object)

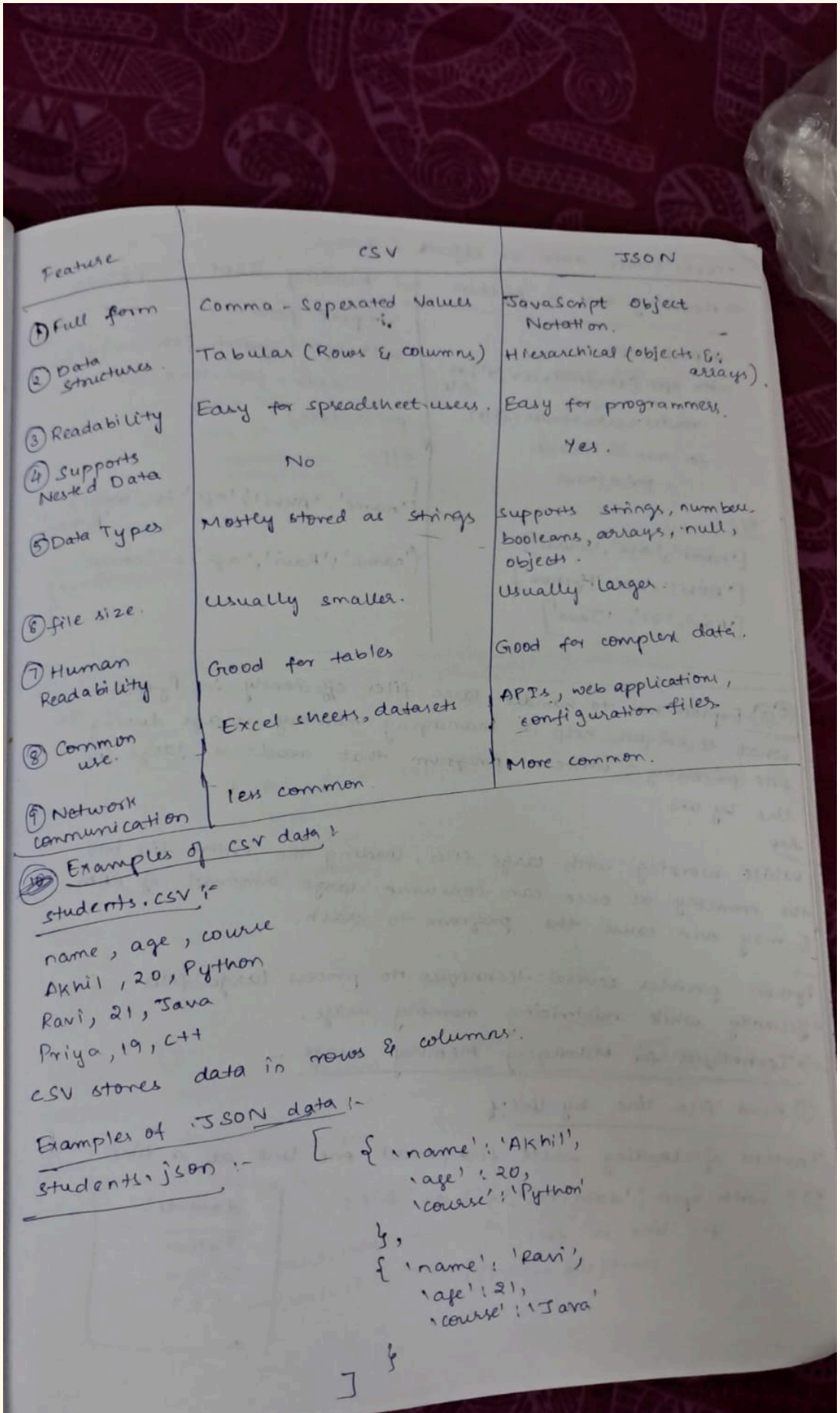
```
ex:- import json
    with open('student.json','r') as file:
        data = json.load(file)

    print(data)
    print(type(data))
```

O/P :-
{'name': 'Akhil', 'age': 20, 'course': 'Python'}
<class 'dict'>

② Compare & contrast the CSV & JSON in python. Which would you prefer one over the other? Provide examples for each format.

Ans :- Both CSV & JSON are popular formats for storing & exchanging data. Python provides built-in modules (csv and json) to work with them.



Feature	CSV	JSON
1. Full form	Comma - Seperated Values	JavaScript Object Notation.
2. Data structures	Tabular (Rows & columns)	Hierarchical (objects & arrays).
3. Readability	Easy for spreadsheet users.	Easy for programmers.
4. Supports Nested Data	No	Yes.
5. Data Types	Mostly stored as strings	Supports strings, numbers, booleans, arrays, null, objects.
6. file size	Usually smaller.	Usually larger.
7. Human Readability	Good for tables	Good for complex data.
8. Common use.	Excel sheets, datasets	APIs, web applications, configuration files.
9. Network communication	less common	More common.

10. Examples of csv data :

students.csv :-

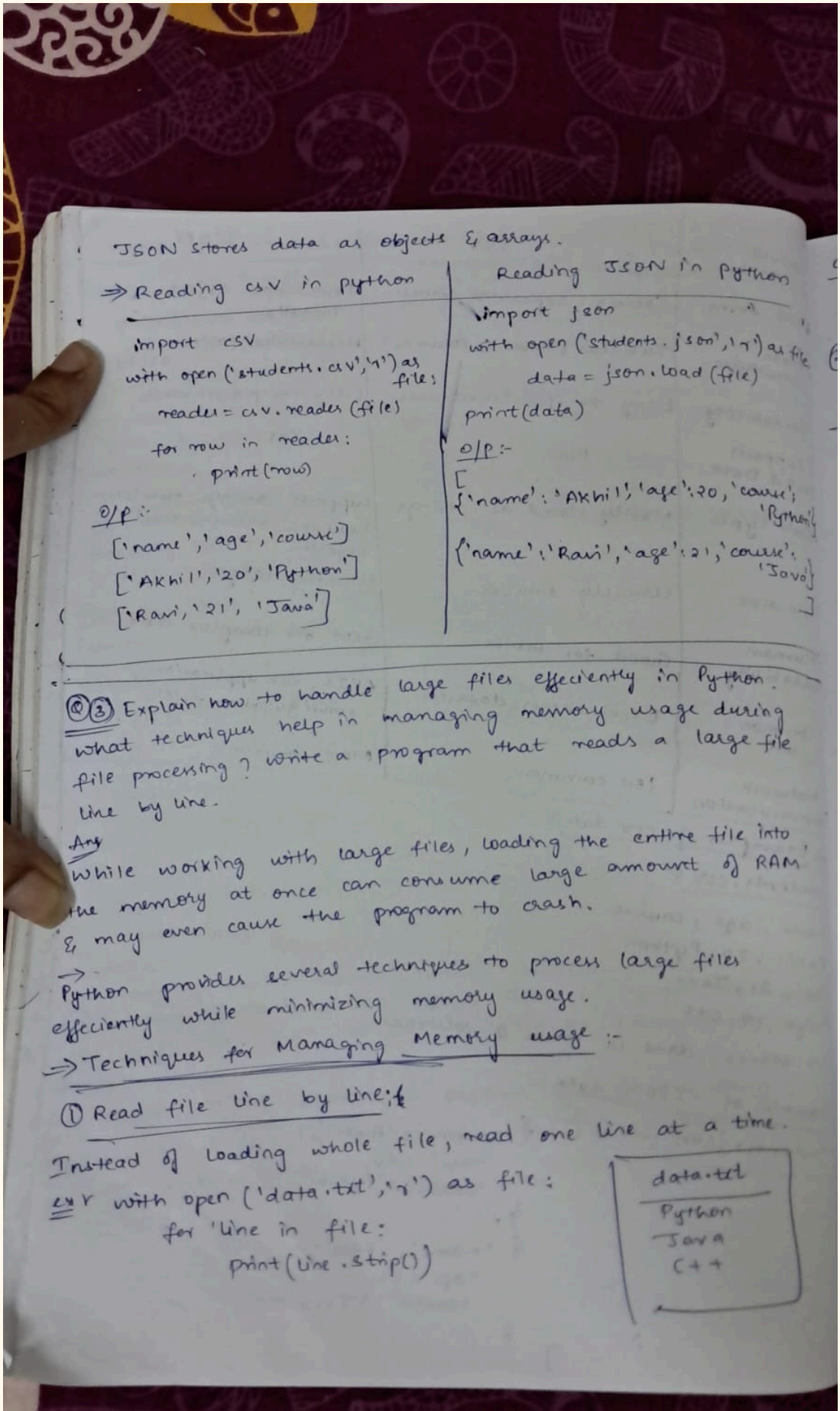
```
name, age, course
Akhil, 20, Python
Ravi, 21, Java
Priya, 19, C++
```

CSV stores data in rows & columns.

Examples of JSON data :-

students.json :-

```
[ { 'name': 'Akhil',
    'age': 20,
    'course': 'Python'
  },
  { 'name': 'Ravi',
    'age': 21,
    'course': 'Java'
  }
]
```



JSON stores data as objects & arrays.

⇒ Reading csv in python

```
import csv
with open('students.csv','r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

O/P :-

```
['name','age','course']
['Akhil','20','Python']
['Ravi','21','Java']
```

Reading JSON in python

```
import json
with open('students.json','r') as file:
    data = json.load(file)
print(data)
```

O/P :-

```
[
{'name':'Akhil','age':20,'course':'Python'}
{'name':'Ravi','age':21,'course':'Java'}
]
```

Q(3) Explain how to handle large files efficiently in Python. What techniques help in managing memory usage during file processing? Write a program that reads a large file line by line.

Ans

While working with large files, loading the entire file into the memory at once can consume large amount of RAM & may even cause the program to crash.

→ Python provides several techniques to process large files efficiently while minimizing memory usage.

⇒ Techniques for Managing Memory usage :-

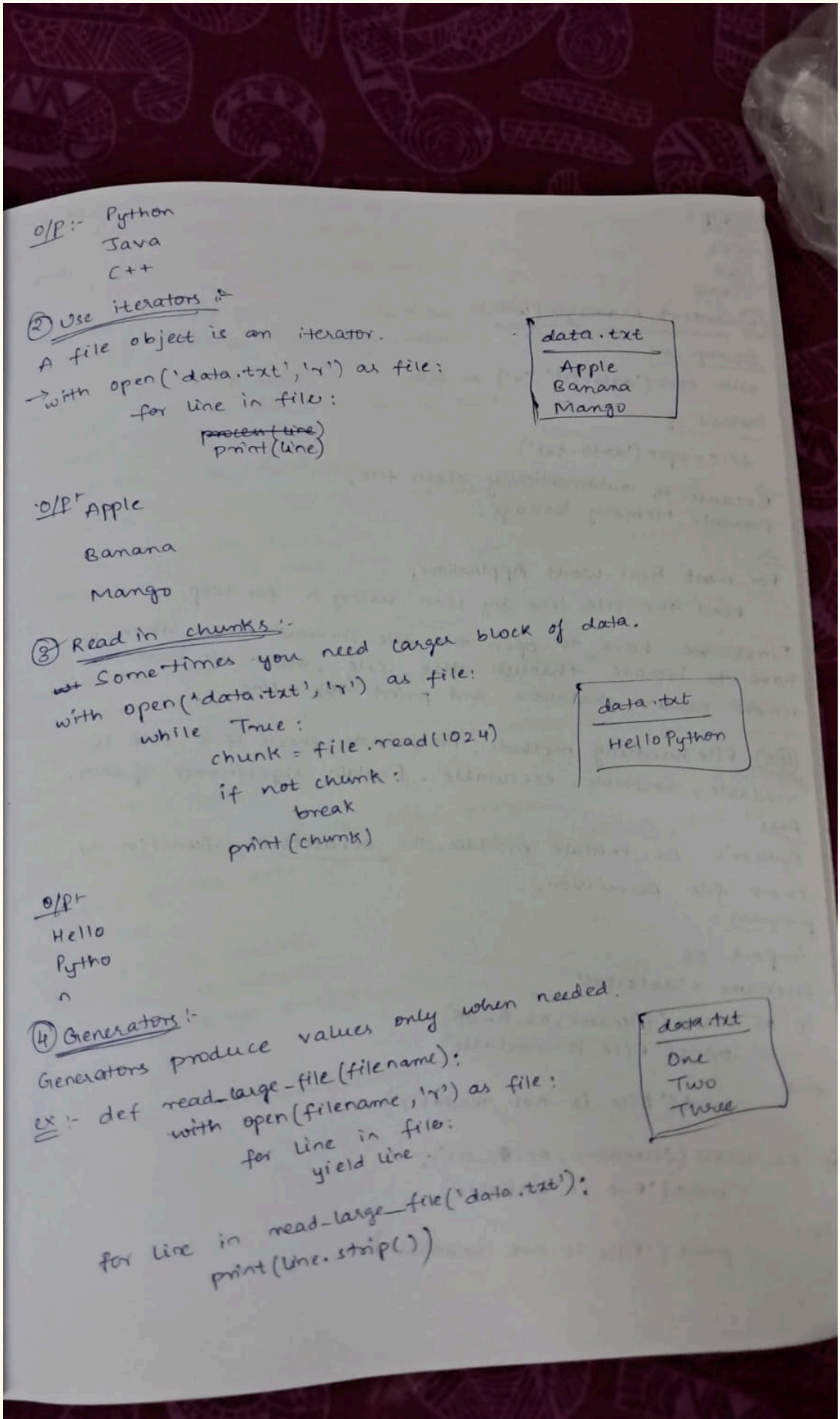
① Read file line by line:

Instead of loading whole file, read one line at a time.

```
with open('data.txt','r') as file:
    for line in file:
        print(line.strip())
```

data.txt

```
Python
Java
C++
```



O/P :- Python
Java
C++

(2) Use iterators :-
A file object is an iterator.
→ with open('data.txt','r') as file:
for line in file:
~~process(line)~~
print(line)

data.txt
Apple
Banana
Mango

O/P :- Apple
Banana
Mango

(3) Read in chunks :-
Sometimes you need larger block of data.
with open('data.txt','r') as file:
while True:
chunk = file.read(1024)
if not chunk:
break
print(chunks)

data.txt
HelloPython

O/P :-
Hello
Pytho
n

(4) Generators :-
Generators produce values only when needed.
ex :- def read_large_file(filename):
with open(filename,'r') as file:
for line in file:
yield line

for line in read_large_file('data.txt'):
print(line.strip())

data.txt
One
Two
Three

O/P :-
One
Two
Three

(5) Context Manager (with) :-

Always use :
with open('data.txt', 'r') as file:

Instead of :
file = open('data.txt')

→ Because it automatically closes file, Prevents Memory leakage.

⇒ For most Real World Applications, Read the file line by line using a for loop.

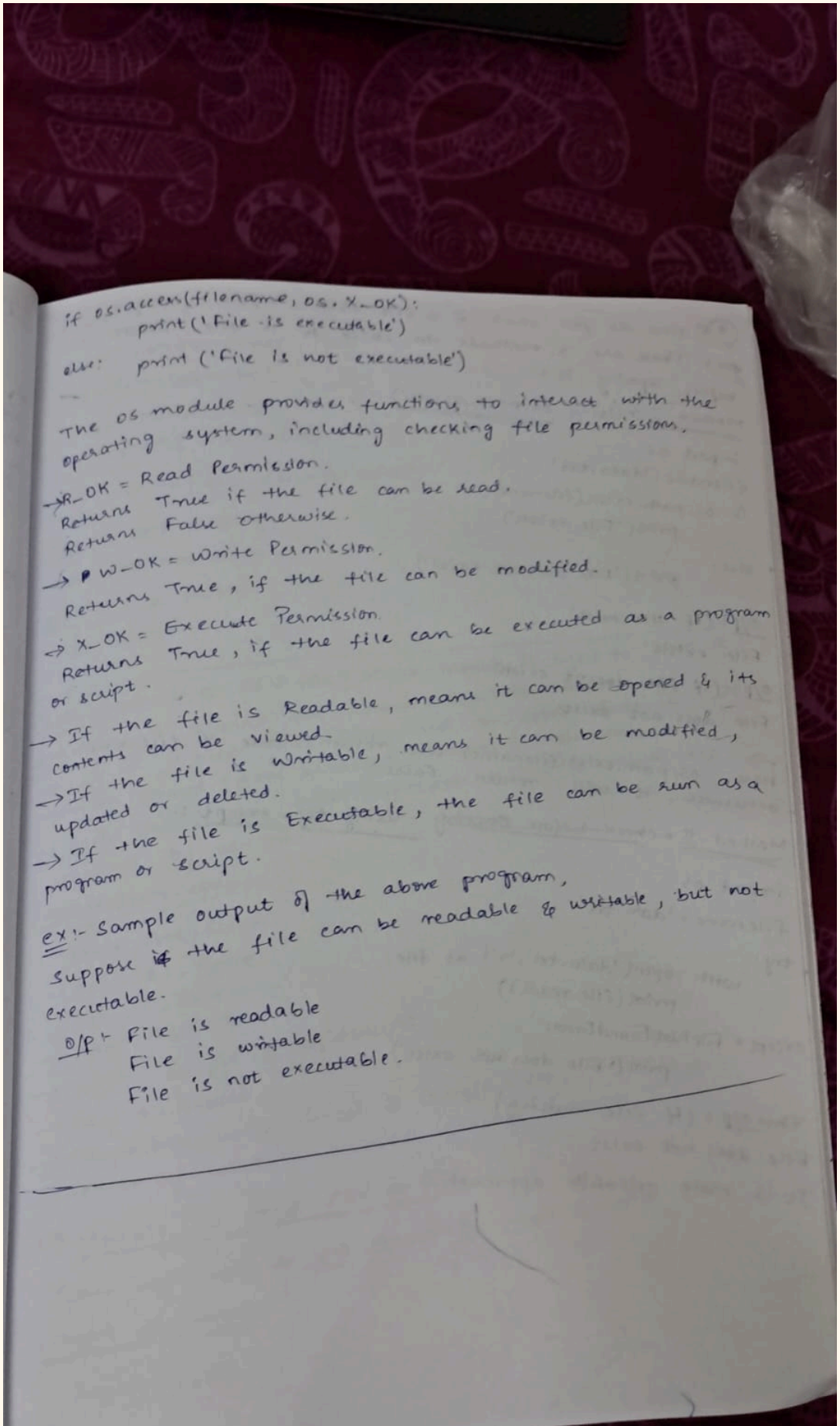
First we have to open the file in read mode, then we have to iterate through the file, then we have to remove newline characters and print the line.

(6)Q) File handling methods. Program to check if a file is readable, writable, executable. Explain significance of each.

Ans:
Python's os module provides the os.access() function to check file permission.

program :-

```
import os
filename = 'data.txt'
if os.access(filename, os.R_OK):
    print('File is readable')
else:
    print('File is not readable')
if os.access(filename, os.W_OK):
    print('File is writable')
else:
    print('File is not writable')
```



```
if os.access(filename, os.X_OK):
        print('File is executable')
else:   print ('File is not executable')
```

The os module provides functions to interact with the operating system, including checking file permissions.

→ R_OK = Read Permission.
Returns True if the file can be read.
Returns False otherwise.

→ W_OK = Write Permission.
Returns True, if the file can be modified.

→ X_OK = Execute Permission.
Returns True, if the file can be executed as a program or script.

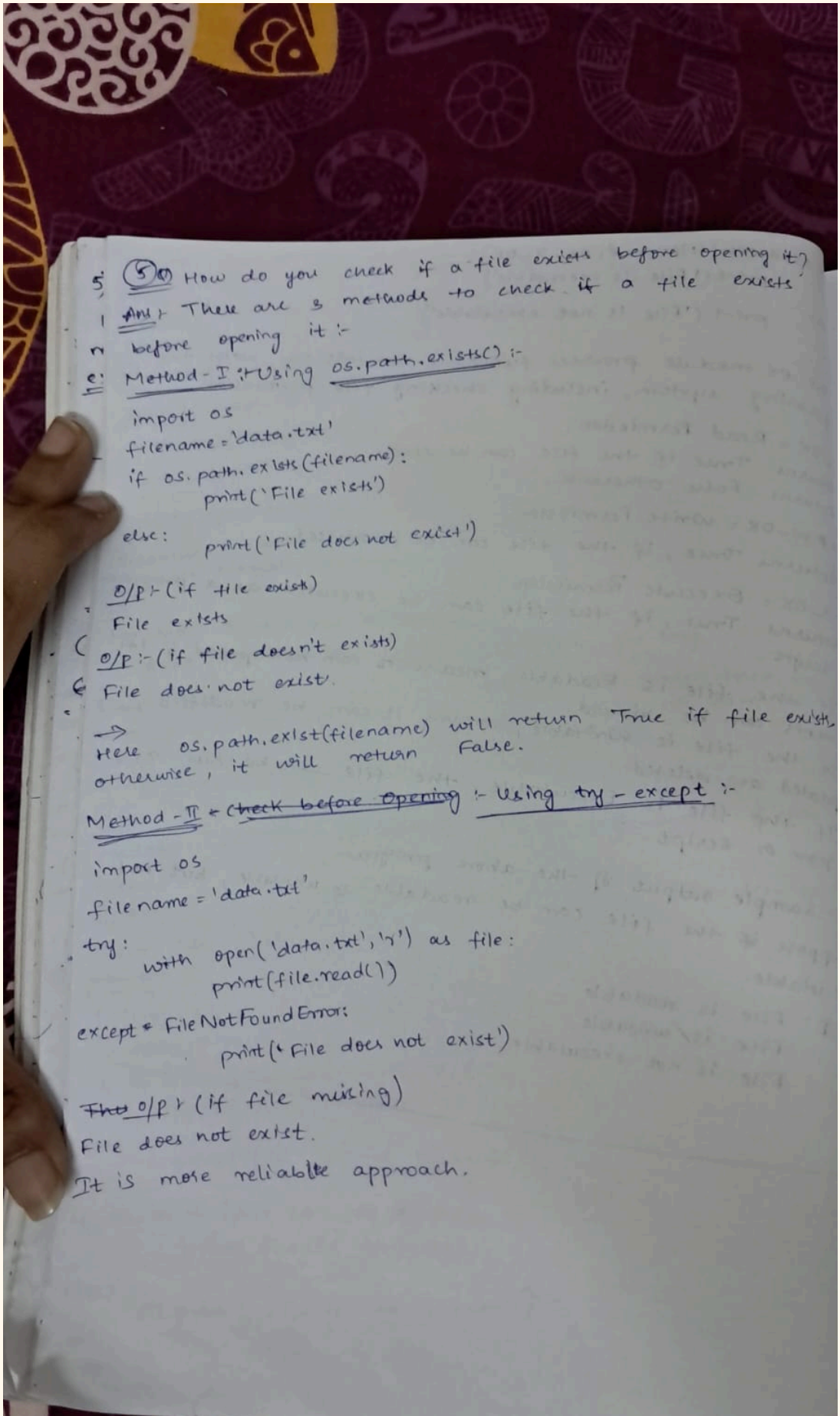
→ If the file is Readable, means it can be opened & its contents can be viewed.

→ If the file is Writable, means it can be modified, updated or deleted.

→ If the file is Executable, the file can be run as a program or script.

ex:- Sample output of the above program,
Suppose the file can be readable & writable, but not executable.

o/p :- File is readable
File is writable
File is not executable.



5) (5) How do you check if a file exists before opening it?

Ans:- There are 3 methods to check if a file exists before opening it :-

Method - I :- Using os.path.exists() :-

```
import os
filename = 'data.txt'
if os.path.exists(filename):
    print('File exists')
else:
    print('File does not exist')
```

O/P :- (if file exists)
File exists

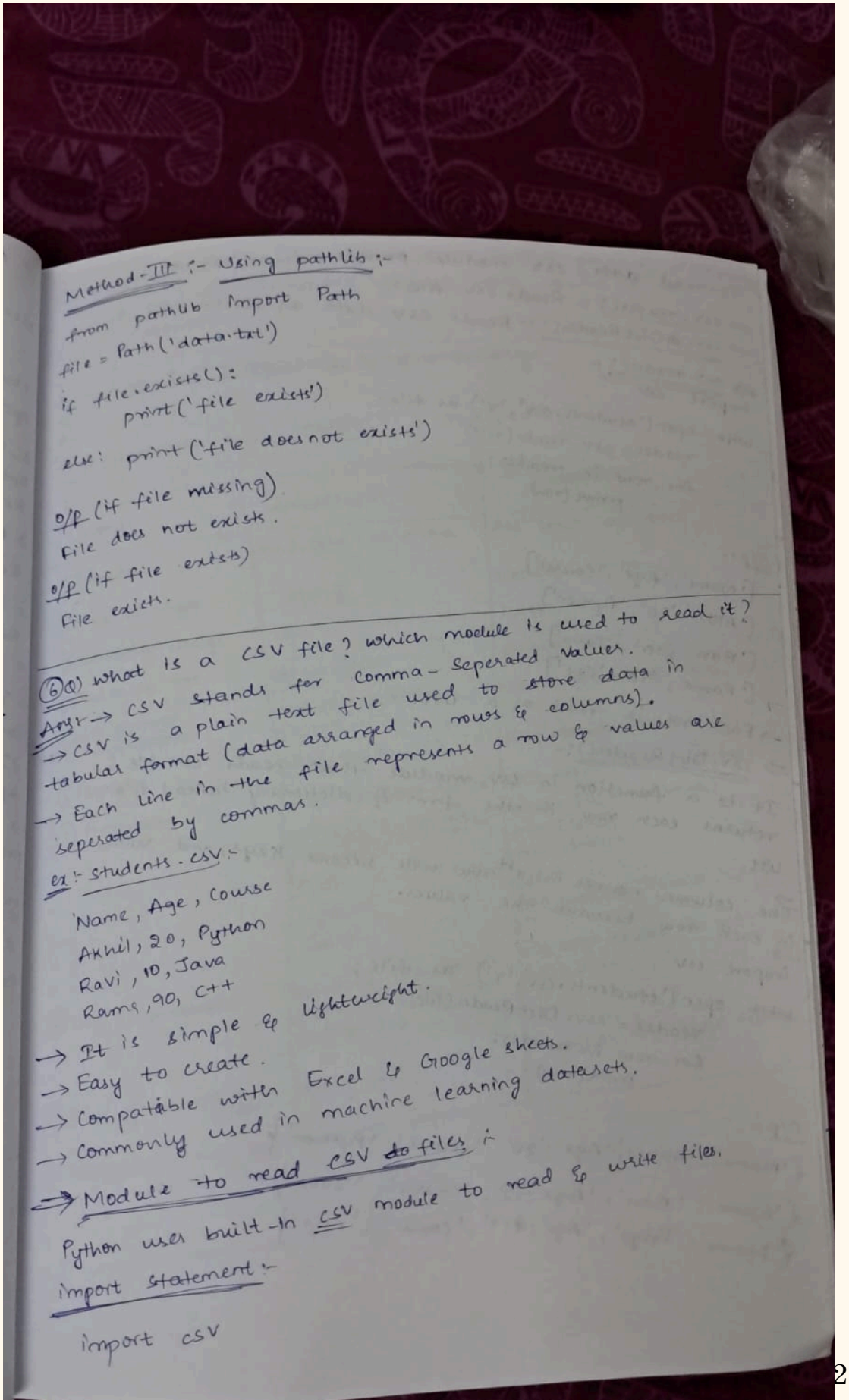
O/P :- (if file doesn't exists)
File does not exist.

→ Here os.path.exist(filename) will return True if file exists, otherwise, it will return False.

Method - II :- ~~Check before Opening~~ Using try - except :-

```
import os
filename = 'data.txt'
try:
    with open('data.txt','r') as file:
        print(file.read())
except FileNotFoundError:
    print('File does not exist')
```

~~The~~ O/P :- (if file missing)
File does not exist.
It is more reliable approach.



Method-II :- Using pathlib :-

```
from pathlib import Path
file = Path('data.txt')
if file.exists():
    print('file exists')
else: print('file does not exists')
```

o/p (if file missing)
File does not exists.

o/p (if file exists)
File exists.

(6@) what is a CSV file? which module is used to read it?

Ans:→ CSV stands for Comma-Seperated Values.
→ CSV is a plain text file used to store data in tabular format (data arranged in rows & columns).
→ Each line in the file represents a row & values are seperated by commas.

ex:- students.csv :-

```
Name, Age, Course
Akhil, 20, Python
Ravi, 10, Java
Rama, 90, C++
```

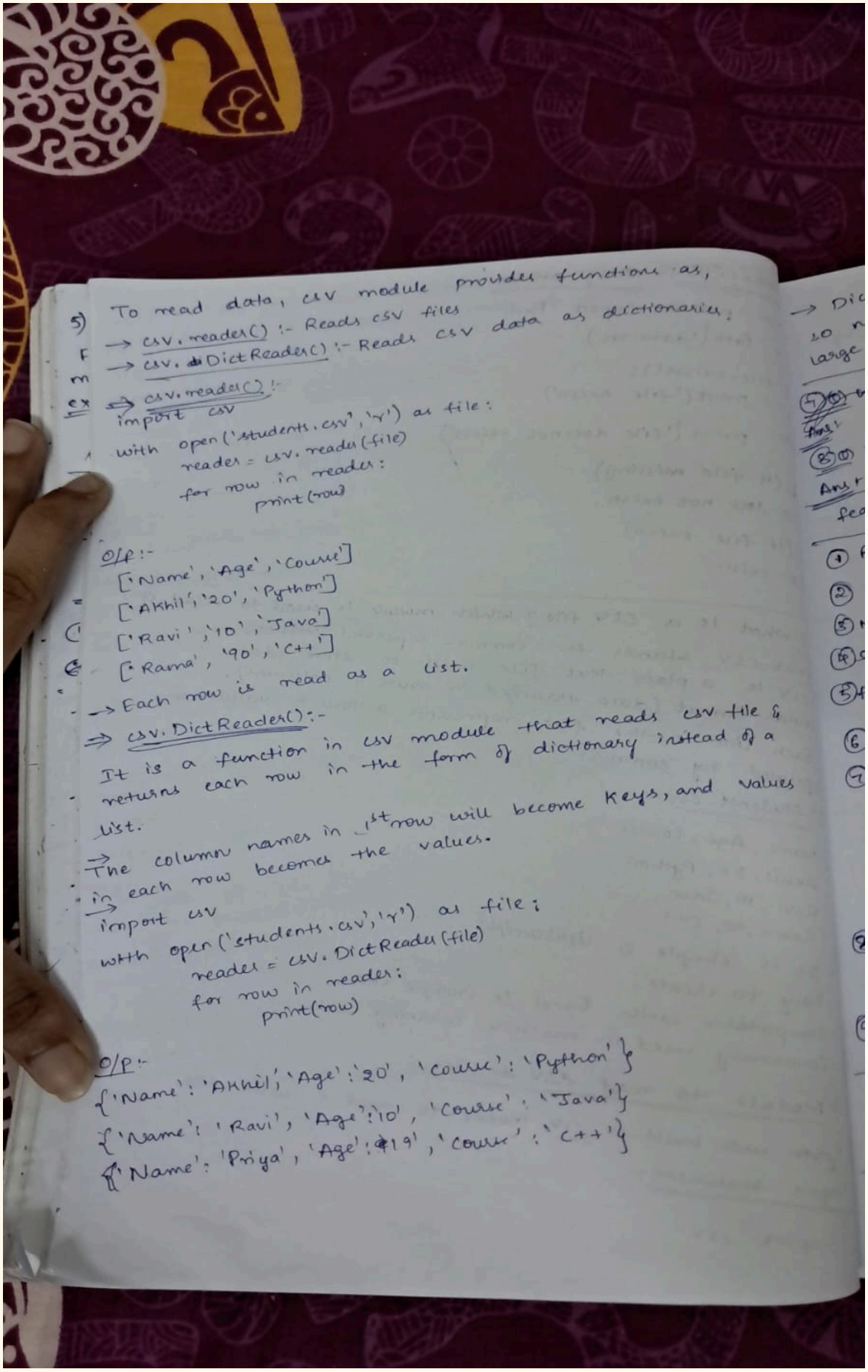
→ It is simple & lightweight.
→ Easy to create.
→ Compatible with Excel & Google sheets.
→ Commonly used in machine learning datasets.

→ Module to read CSV ~~do~~ files :-

Python uses built-in csv module to read & write files.

import statement :-

```
import csv
```



5) To read data, csv module provides functions as,
→ csv.reader() :- Reads csv files
→ csv.DictReader() :- Reads csv data as dictionaries.

⇒ csv.reader() :-

```
import csv
with open('students.csv','r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

O/P :-

```
['Name', 'Age', 'Course']
['Akhil', '20', 'Python']
['Ravi', '10', 'Java']
['Rama', '90', 'C++']
```

→ Each row is read as a list.

⇒ csv.DictReader() :-

It is a function in csv module that reads csv file & returns each row in the form of dictionary instead of a list.

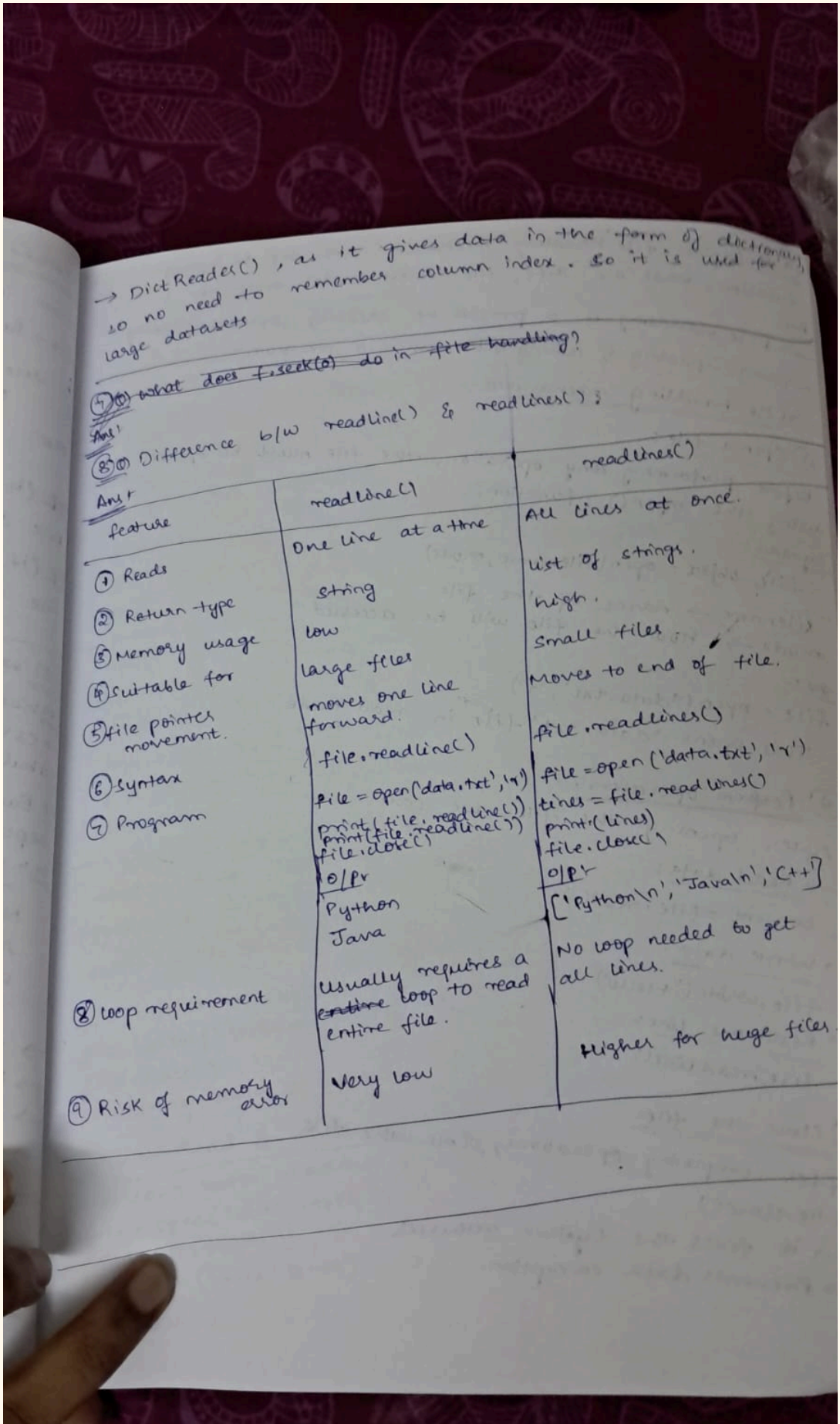
→ The column names in 1st row will become keys, and values in each row becomes the values.

```
import csv
with open('students.csv','r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)
```

O/P :-

```
{'Name': 'Akhil', 'Age': '20', 'Course': 'Python'}
{'Name': 'Ravi', 'Age': '10', 'Course': 'Java'}
{'Name': 'Priya', 'Age': '19', 'Course': 'C++'}
```

Back



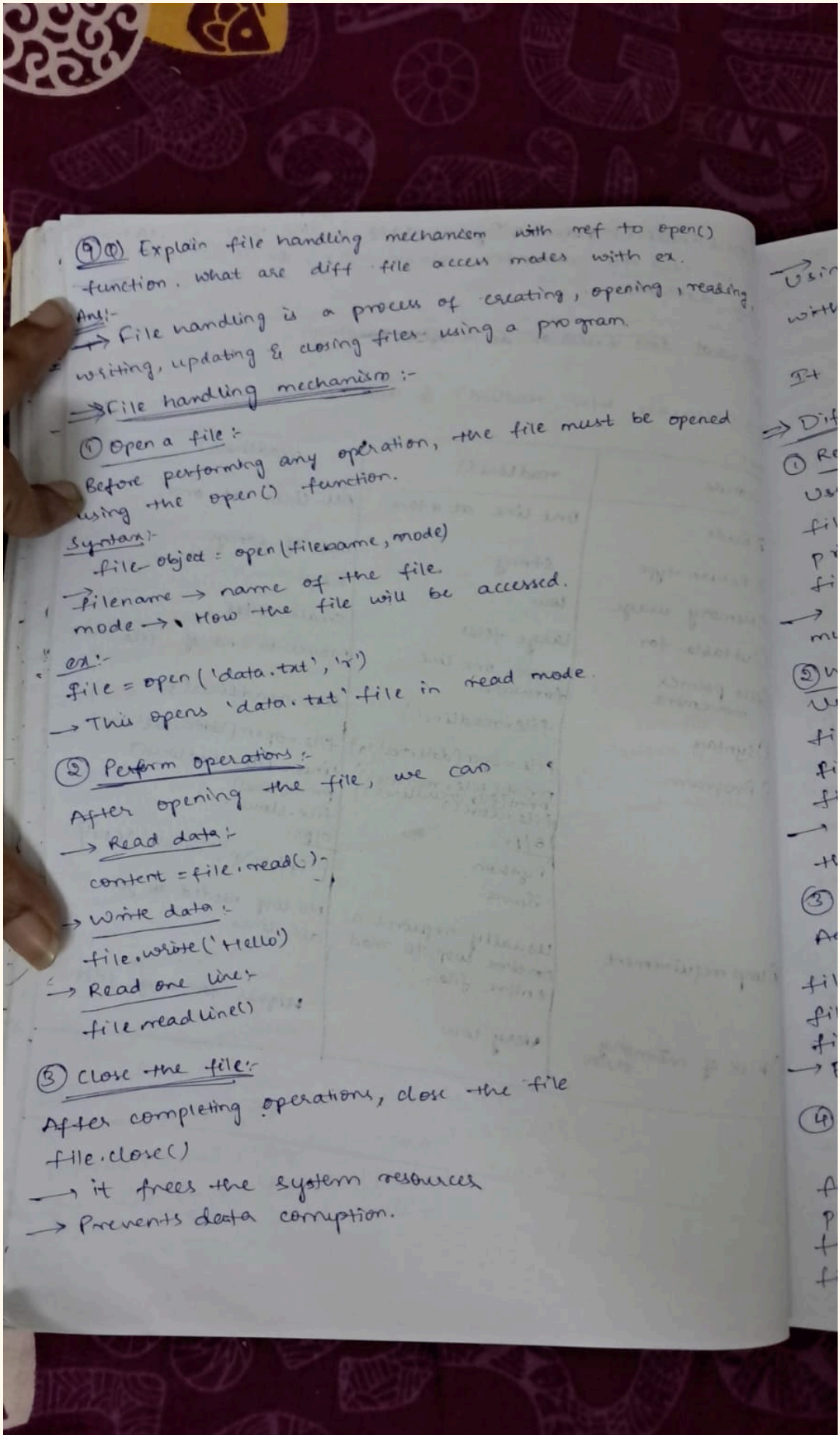
→ DictReader() , as it gives data in the form of dictionary, so no need to remember column index. So it is used for large datasets

~~(7)(a) what does f.seek(0) do in file handling?~~

Ans:

(8)(a) Difference b/w readline() & readlines() :

Feature	readline()	readlines()
(1) Reads	One line at a time	All lines at once.
(2) Return type	string	list of strings.
(3) Memory usage	low	high.
(4) Suitable for	large files	small files.
(5) file pointer movement.	moves one line forward.	Moves to end of file.
(6) Syntax	file.readline()	file.readlines()
(7) Program	file = open('data.txt','r') print(file.readline()) print(file.readline()) file.close() O/P: Python Java	file = open('data.txt','r') lines = file.readlines() print(lines) file.close() O/P: ['Python\n','Java\n','C++']
(8) loop requirement	Usually requires a ~~entire~~ loop to read entire file.	No loop needed to get all lines.
(9) Risk of memory error	Very low	Higher for huge files.



(9) (a) Explain file handling mechanism with ref to open() function. what are diff file access modes with ex.

Ans:-

→ File handling is a process of creating, opening, reading, writing, updating & closing files using a program.

→ File handling mechanism :-

① Open a file :-

Before performing any operation, the file must be opened using the open() function.

Syntax :-

file_object = open(filename, mode)

→ filename → name of the file.

mode → How the file will be accessed.

ex :-

file = open('data.txt', 'r')

→ This opens 'data.txt' file in read mode.

② Perform operations :-

After opening the file, we can

→ Read data :-

content = file.read()

→ Write data :-

file.write('Hello')

→ Read one line :-

file.readline()

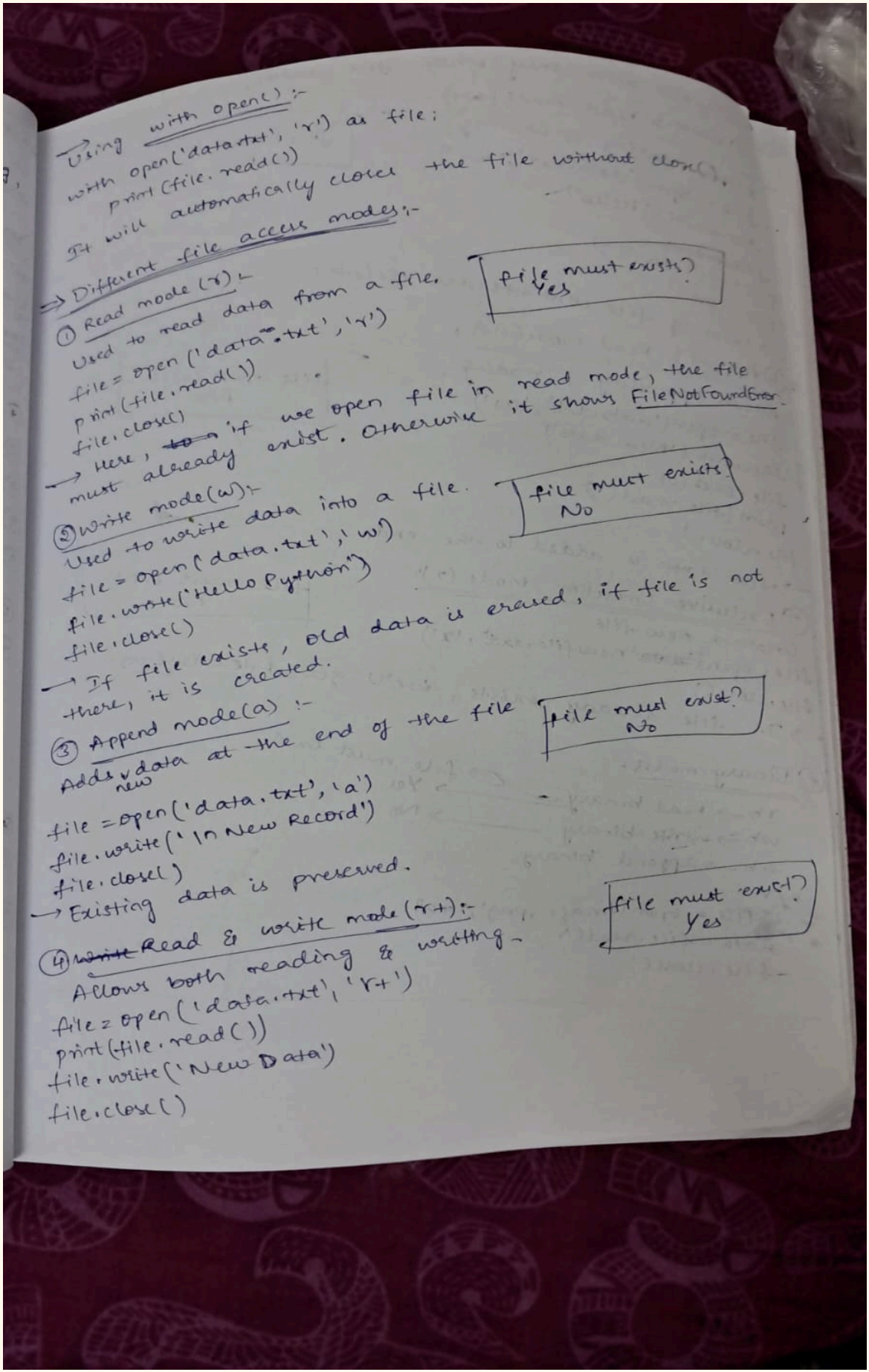
③ Close the file :-

After completing operations, close the file

file.close()

→ it frees the system resources

→ Prevents data corruption.



→ Using with open() :-

```
with open('data.txt', 'r') as file:
    print(file.read())
```

It will automatically closes the file without close().

→ Different file access modes :-

① Read mode (r) :-

Used to read data from a file.

file must exists?
Yes

```
file = open('data.txt', 'r')
print(file.read())
file.close()
```

→ Here, if we open file in read mode, the file must already exist. Otherwise it shows FileNotFoundError.

② Write mode (w) :-

Used to write data into a file.

file must exists?
No

```
file = open('data.txt', 'w')
file.write('Hello Python')
file.close()
```

→ If file exists, old data is erased, if file is not there, it is created.

③ Append mode (a) :-

Adds new data at the end of the file

file must exist?
No

```
file = open('data.txt', 'a')
file.write('In New Record')
file.close()
```

→ Existing data is preserved.

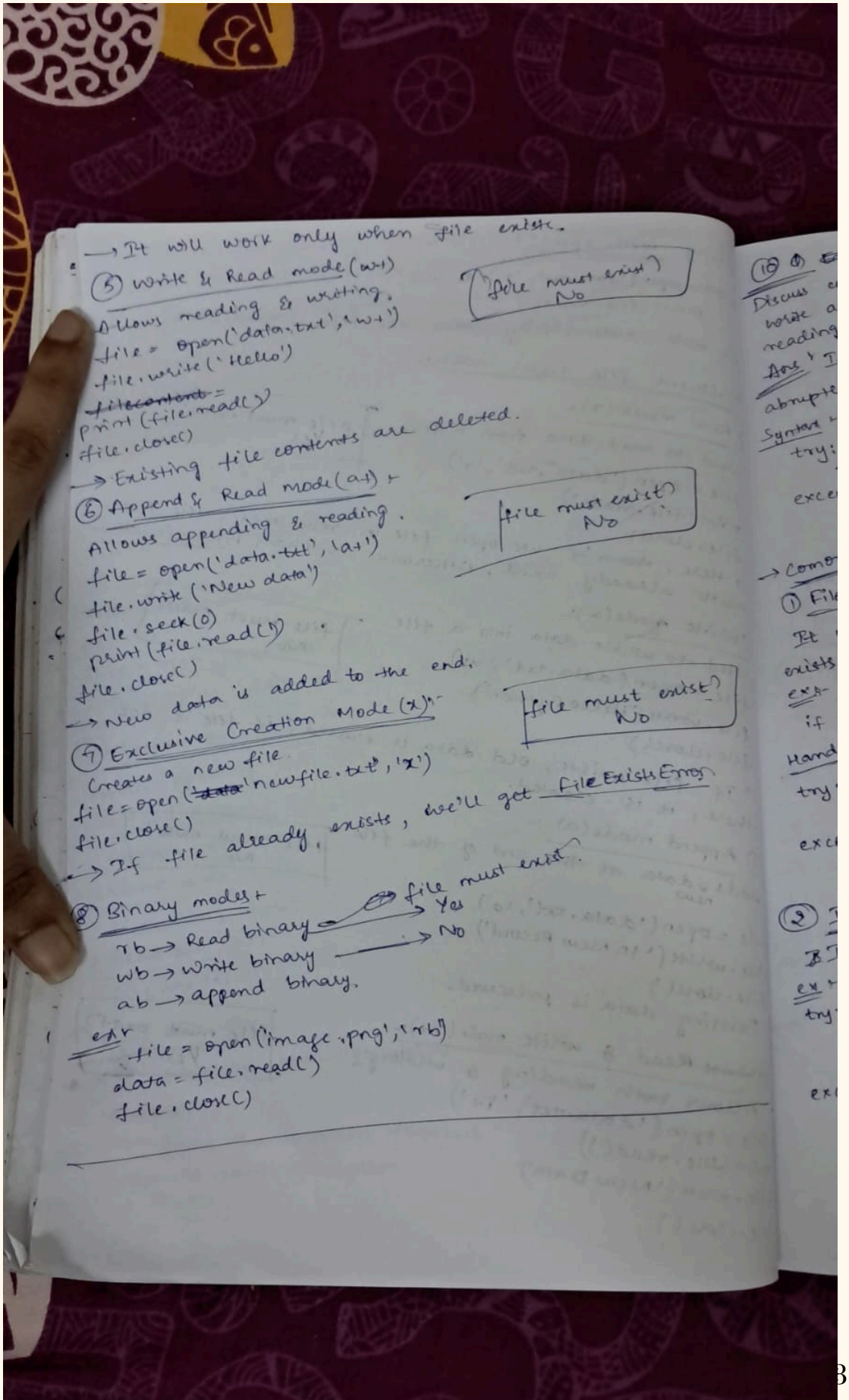
④ ~~write~~ Read & write mode (r+) :-

Allows both reading & writting.

file must exist?
Yes

```
file = open('data.txt', 'r+')
print(file.read())
file.write('New Data')
file.close()
```

Back



→ It will work only when file exists.

(5) Write & Read mode (w+)
Allows reading & writing.
file = open('data.txt', 'w+')
file.write('Hello')
~~filecontent~~ =
print(file.read())
file.close()
→ Existing file contents are deleted.

file must exist?
No

(6) Append & Read mode (a+)
Allows appending & reading.
file = open('data.txt', 'a+')
file.write('New data')
file.seek(0)
print(file.read())
file.close()
→ New data is added to the end.

file must exist?
No

(7) Exclusive Creation Mode (x)
Creates a new file
file = open(~~'data'~~ 'newfile.txt', 'x')
file.close()
→ If file already exists, we'll get FileExistsError

file must exist?
No

(8) Binary modes
rb → Read binary → file must exist? Yes
wb → write binary → No
ab → append binary.

ex
file = open('image.png', 'rb')
data = file.read()
file.close()